

dr. Matej Šprogar

# Razred

Preprosto delo s kompleksnimi podatki



# Uvod

Poskusimo uporabiti najpreprostejši kompleksen podatek (tak, ki v sebi združuje dve celi števili). Do neke točke bo vse izvedljivo tudi z našim trenutnim znanjem programiranja, ampak počasi se bo pričela koda zapletati: ko bo mnogo takšnih podatkov, ko bodo funkcije zahtevale več argumentov...

To vse povečuje verjetnost programerske napake. Prevajalnik nam ne more pomagati, saj vse, kar vidi, so cela števila in dokler ne naredimo nekaj *nelegalnega* s celimi števili...

# Točka v ravnini

Navadna 2D točka ima celoštevilski koordinati  $x$  in  $y$ . Točko  $T$  znamo definirati kot dve ločeni in s tem neodvisni spremenljivki  $T_x$  in  $T_y$ ; enako velja za točko  $U$ . Za človeškega bralca programa so **nujni** dodatni komentarji (ki prevajalniku ne koristijo).

Funkcija, ki pričakuje eno točko (ali več), rabi več kot en argument. Sicer logično, a dolgovezno.

Ker sta imeni točk  $T$  in  $U$  prevajalniku neznani, ju ne dovoli uporabiti v kodi.

```
int Tx, Ty;           // točka T
int Ux, Uy;           // točka U
```

Funkcija za izračun razdalje rabi **štiri** argumente (čeprav rabi **dve** točki):

```
double razdalja( int Ax, int Ay,
                  int Bx, int By) {}
```

*Verjetnost*, da jo bomo uporabili narobe, je večja, kot če bi zahtevala zgolj 2 točki:

```
razdalja(Tx, Tx, Ux, Uy); // NAROBE
razdalja(T, U);           // syntax error 😞
```

# Poskus s poljem

Prevajalniku namreč nismo povedali, da ime *T* označuje dve spremenljivki, *Tx* in *Ty*, ki spadata **skupaj**.

Pa ju "združimo" s poljem - namesto dveh ločenih *intov* za eno točko, lahko ustvarimo eno polje z dvema *intoma*, ki predstavljata par (*int*, *int*).

Pristop deluje le, če združimo podatke **istega tipa**. Ko temu ni tako, polja ni mogoče ustvariti. Recimo cene izdelkov na tržnici so pari (*ime\_zivila*, *cena\_zivila*), torej (*string*, *double*).

```
int[] T = new int[2]; // točka T(0,0)
int[] U = {3, 4};      // točka U(3,4)
```

```
preveri(5 == razdalja(T, U));
```

je legalna in delujoča koda, če le imamo

```
double razdalja(int[] A, int[] B)
{
    predpogoj(A.Length==2 && B.Length==2);
    int dx = A[0] - B[0]; // X na indexu 0
    int dy = A[1] - B[1]; // ročno delo! 😞
    return Math.Sqrt(dx*dx + dy*dy);
}
```

# Naivni poskus

Če želimo vseeno uporabiti polja za recimo cene na tržnici, moramo uporabiti več polj. Izvedljivo, vendar delamo vse ročno z indeksi...

Uporaba ni enostavna:

```
lepo_izpisi("sir", 4.20) ali  
lepo_izpisi(2, imena, cene);
```

Če pa želimo npr. dodati številko stojnice, moramo popraviti **vso** kodo!

Prevajalniku bi želeli povedati, da string in double **skupaj** predstavljata eno Zivilo...

```
string[] imena = {"zelje", "krompir", "sir"};  
double[] cene = {0.90, 0.80, 4.20};
```

```
int id_naj = isci_najdrazje(imena, cene); //  
lepo_izpisi(imena[id_naj], cene[id_naj]); //
```



## Mnogo bolj berljivo bi bilo (recimo):

```
Zivilo[] zaloga = {  
    ["zelje", 0.90] ,           // JS*, Python*  
    Zivilo("krompir", 0.80),    // C++  
    new Zivilo("sir", 4.20)     // Java, C#  
};
```

```
Zivilo naj = isci_najdrazje(zaloga); //  
lepo_izpisi(naj); //
```



## Ampak kaj je Zivilo?

# Definicija novega podatkovnega tipa

Spremenljivke v razrednem **bloku** (tim. atributi (Java), fieldi (C#), member variable (C++)) skupaj določajo **podtis** podatkov v pomnilniku.

Za dostop do podatka rabimo piko:

**referenca.ime\_atributa**

`sir.ime`

Primerku konkretnih podatkov v pomnilniku pravimo tudi **objekt** ali instanca razreda.

```
class Zivilo {  
    string ime;  
    double cena;  
}
```

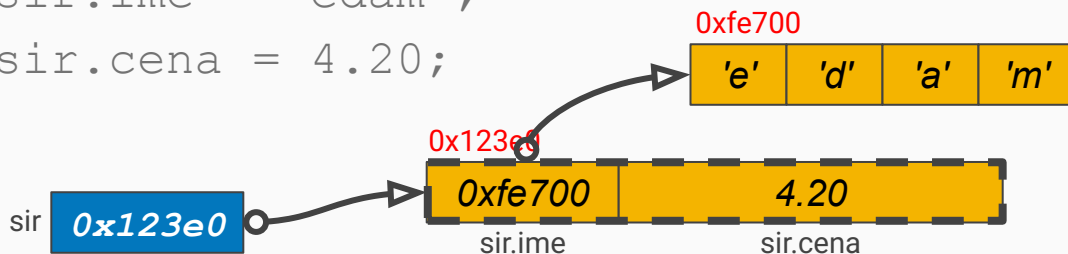
// atribut/field

tip

referenca

primerek

```
Zivilo sir = new Zivilo(); //Java, C#  
sir.ime = "edam";  
sir.cena = 4.20;
```



# Razred (*class*)

Večina jezikov omogoča definicijo **lastnega** (uporabniškega) podatkovnega tipa, ki bo *enakovreden*\* vgrajenim tipom. Tipičen predstavnik *uporabniškega* tipa je recimo seznam (`ArrayList<>` v Javi, `List<>` v C# in `vector<>` v C++), medtem ko je tip `string` v Javi in C# na tanki meji obeh svetov.

Definicija razreda obsega dve osnovni komponenti - **podatke** in **funkcije** (C# uvaja še hibridno *lastnost* (*property*)). Na podlagi *podatkov* v definiciji našega tipa potem prevajalnik določi potrebno količino pomnilnika.

# Neodvisni objekti

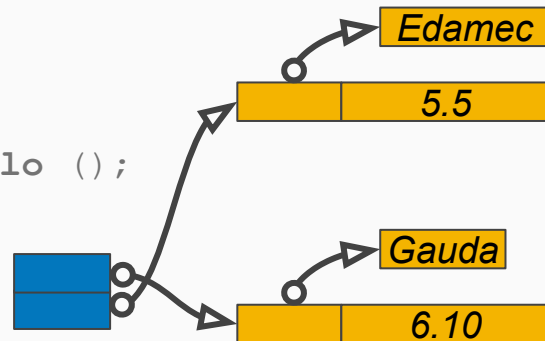
Definicija razreda je "predloga", s katero lahko operator **new** naredi nove in nove objekte našega tipa.

Podobno nalogo ima risalna šablona, je pa je beseda *šablona* (*template*) v C++ rezervirana za nekaj drugega...

Razred (npr. `Zivilo`) je pravopisno navodilo za izdelavo poljubno mnogo različnih objektov. Na primeru desno imamo v pomnilniku dva objekta in dve referenci: `sir` in `cheese`.

```
Zivilo sir = new Zivilo ();  
sir.ime = "Edamec";  
sir.cena = 5.50;
```

```
Zivilo cheese = new Zivilo ();  
cheese.ime = "Gauda";  
cheese.cena = 6.10;
```



Operator `new` je v dveh klicih uporabil **isti** recept in naredil 2 nova in **neodvisna** objekta - vsakega s *svojim* koščkom spomina in svojimi podatki.

Deklaracija enega razrednega atributa/fielda v okviru razreda povzroči po eno variabla v vsakem ustvarjenem objektu.



# Razred lahko vsebuje metode

Beseda *objekt* dobi svoj polni smisel šele, ko primerke nekega razreda dodatno opremimo še s funkcijami, ki lahko **edine** uporabljajo ta objekt.

Na ta način prevajalniku povemo, kaj je sploh legalno delati z objekti (recimo seštevati, ne pa tudi odštevati stringe). Do funkcije dostopamo podobno kot do atributa; funkcija "vidi" podatke objekta kot "**svoje**" - ne rabimo jih navajati med argumenti!

Funkcijo v sklopu razreda imenujemo tudi (razredna) **metoda**.

```
class Zivilo {  
    string ime;  
    double cena;  
    void print() {  
        Console.WriteLine(ime + ' ' + cena);  
    }  
}
```

```
Zivilo sir = new Zivilo();  
sir.ime = "edamec";  
sir.cena = 5.50;  
  
sir.print();
```

Objektu *sir* smo ukazali, naj se sprinta. Kot objekt pozna "**svoje**" ime in ceno...

# Statične vsebine

Ko v razredu deklariramo razredni atribut/field, bo vsak objekt tega tipa dobil **svoj** atribut - objekt nima dostopa do atributov drugih objektov!

Včasih pa želimo, da bi si vsi objekti delili ISTO spremenljivko, da bi torej lahko VSI objekti uporabljali ISTO vrednost. Kar naredi en, "vidijo" vsi...

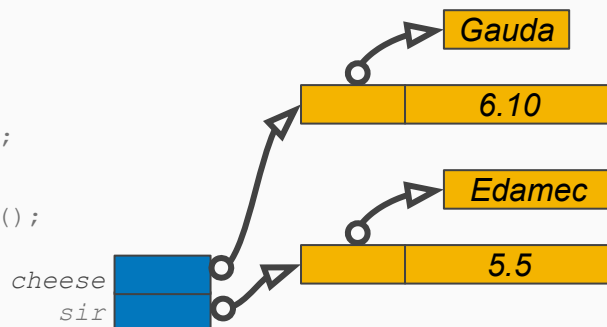
Takšne attribute označimo kot statične (`static`) in prevajalnik jih ne bo shranil v prostor objekta, ampak ločeno.

```
class Zivilo {  
    string ime;  
    double cena;  
    static int stevec = 0;  
    static void stej() { stevec += 1; }  
    void print() {  
        Console.WriteLine(stevec);  
    }  
}
```



```
Zivilo sir = new Zivilo ();  
sir.ime = "Edamec";  
sir.cena = 5.50;  
Zivilo.stej();  
preveri(Zivilo.stevec == 1);
```

```
Zivilo cheese = new Zivilo ();  
cheese.ime = "Gauda";  
cheese.cena = 6.10;  
Zivilo.stej();  
preveri(Zivilo.stevec == 2);
```



# Statična ali navadna metoda?

Statična metoda je skupna vsem objektom, nima torej "svojega" objekta, nad katerim bi bila klicana. Zato lahko statična metoda *enostavno* dostopa zgolj do statičnih atributov razreda, sicer rabi referenco. Java in C# zahtevata pri zunanjem klicu še ime razreda (*Tip.metoda()*),

Navadna metoda pa ima tim. **this** referenco, ki je razvidna iz klica in metodi omogoča enostaven dostop do *lastnih* razrednih spremenljivk; za dostop do vsebine *drugega* objekta potrebuje njegovo referenco.

```
class Oseba {  
    string ime;  
  
    void preimenuj(string novo_ime) {  
        this.ime = novo_ime; // this je "skriti" argument  
        ime = novo_ime;      // enakovreden krajši zapis  
    }  
    static void rename(Oseba x, string novo_ime) {  
        x.ime = novo_ime;    // nimamo this, imamo x  
    }  
}  
  
Oseba kekec = new Oseba();  
kekec.ime = "Kekec";
```

Sedaj lahko izdamo dva **enakovredna** ukaza, razlika med njima je zgolj "lepotne" narave:

```
kekec.preimenuj("Rožle");    // objektna sintaksa  
rename(kekec, "Rožle");      // funkcijska sintaksa
```

# Java nima struct, C# struct pa je med Javo in C++ struct...

Java (zaenkrat\*) omogoča le izdelavo referenčnih tipov z rezervirano besedo **class**.

C# ima dodatno še hibridni **struct**, ki omogoča izdelavo (omejenih) vrednostnih tipov.

V C++ sta oba, *class* in *struct*, vrednostna tipa.

```
struct Struktura {           // C#, C++
    public int vrednost;
}
class Razred {              // Java, C#, C++
    public int vrednost;
}
Razred x = new Razred();    //Java, C#
Razred x;                   //C++
```

```
int A = 42, AA = A;
AA = 0;
// C# Java C++: A == 42, AA == 0
```

```
string B = "42", BB = B;
BB = "0";
// C# Java C++: B == "42"
```

```
Struktura C = new Struktura();
C.vrednost = 42;
Struktura CC = C;
CC.vrednost = 0;
// C#: C.vrednost == 42
```

```
Razred D = new Razred();
D.vrednost = 42;
Razred DD = D;
DD.vrednost = 0;
// C#, Java: D.vrednost == 0
```

# Vidljivost kode

Podatke in metode objekta lahko tudi *skrijemo* pred **zunanjo** (tujo) kodo (programsko kodo **izven** našega razreda; tu ni mišljena zlonamerna, ampak "dobronamerna" koda, ki jo lahko napišemo celo mi sami).

Če to NI problem, jih lahko označimo za javno uporabo (*public*). Privzeto pa je vsa vsebina zasebna (*private*).

**"Tuja" koda bi naj delala s privatno vsebino zgolj preko javnih metod!**

Več o tem naslednji semester!

```
class Boeing767 {  
    private void ustavi() { hkrati(levi, desni, zaviraj); }  
    public Motor levi, desni;  
  
    public void vkrcaj() { ... }  
    public void vzleti() { ... }  
    public void pristani() {  
        ... // pozicija, podvozje, zakrilca, hitrost, ...  
        ustavi();  
    }  
}  
  
class LaudaAir {  
    public static void main(String[] args) {  
        Boeing767 mozart = new Boeing767();  
        mozart.vkrcaj();  
        mozart.vzleti();  
  
        // naivno pristajanje  
        mozart.levi.zaviraj(); // LEGALNO ampak SMRTNO  
        mozart.ustavi();      // compile error  
    }  
}
```

# Še o referencah in objektih v Javi in C#

Enako, kot rabimo referenco za polja, rabimo reference tudi za objekte. Posledično lahko isti objekt uporabimo preko več enakih referenc. Za izdelavo reference NE SMEMO uporabiti operatorja *new*, zadostuje deklaracija!

Pri referenčnih tipih tako velja:  
deklaracija naredi referenco, **new** pa objekt!

Pri vrednostnih tipih pa velja:  
deklaracija naredi objekt (spremenljivko)!

neumno je ukazati recimo

```
Hisa moja = new Hisa();  
Hisa tvoja = new Hisa();  
tvoja = moja;
```



ker smo po nepotrebnem ustvarili drugo hišo!  
Pravilno je

```
Hisa tvoja;  
tvoja = moja;
```

ali v enem ukazu

```
Hisa tvoja = moja;
```

# Kako govorimo o objektih...

Objekt v pomnilniku ustvarimo z operatorjem *new*, referenco pa moramo ohraniti za kasnejši dostop do njega. Ime reference "postane" ime objekta:

```
Tip referenca_objekta = new Tip();  
AlbertKeks albert = new AlbertKeks();
```

V Javi in C# je enostavneje, ampak narobe, reči "objekt albert" kot pa (pravilno) "objekt, ki ga nadzoruje referenca albert"; torej v človeški komunikaciji izenačimo referenco z objektom, čeprav to dejansko ni res.